Cahier des charges Projet M1 SYRUIS

Génération automatique de graphes de personnages à partir d'un corpus littéraire

1. Contexte et Objectifs du Projet

1.1. Contexte

Ce projet s'inscrit dans le domaine du traitement automatique du langage naturel (TALN). Il consiste à développer un programme permettant d'identifier automatiquement les personnages d'un corpus littéraire, de reconnaître leurs différentes mentions dans le texte et de construire des graphes représentant leurs relations.

Le corpus principal est constitué de textes issus du cycle *Fondation* d'Isaac Asimov. Les documents sont initialement fournis au format PDF. Leur exploitation nécessite plusieurs étapes de prétraitement, car l'extraction du texte peut introduire des coupures de mots, des espaces incorrects ou d'autres erreurs.

Le projet comporte deux tâches complémentaires :

- **Tâche 1 Analyse du corpus principal :** préparer les textes, extraire les entités candidates, identifier les personnages et construire des graphes représentant leurs interactions dans les livres étudiés.
- **Tâche 2 Compétition Kaggle :** développer et améliorer un algorithme capable de générer automatiquement un graphe de personnages pour chaque chapitre d'un corpus d'évaluation. Les graphes devront respecter le format demandé par la compétition et être soumis sur Kaggle afin d'obtenir un score F1 aussi élevé que possible.

L'objectif général est de passer d'un texte brut à un réseau de personnages exploitable, tout en limitant les erreurs de reconnaissance et de construction des relations.

1.2. Objectifs

Les objectifs du projet sont les suivants :

- Extraire le texte des documents PDF et préparer le corpus pour l'analyse.
- Nettoyer et tokenizer les textes.
- Calculer les fréquences des mots et des séquences de mots.
- Générer une liste préliminaire d'entités candidates.
- Filtrer les candidats pour identifier les personnages et, si nécessaire, les lieux.
- Reconnaître les mentions de personnages dans le texte.

- Regrouper les différentes appellations d'un même personnage, par exemple « Hari » et « Hari Seldon ».
- Détecter les interactions entre personnages à partir de leurs cooccurrences dans le texte.
- Construire des graphes de personnages pour les livres du corpus principal.
- Générer un graphe distinct pour chaque chapitre du corpus d'évaluation.
- Exporter les graphes au format GraphML et préparer le fichier CSV attendu pour Kaggle.
- Évaluer les résultats avec les scores F1 des nœuds et des arêtes.
- Améliorer progressivement les algorithmes afin d'obtenir un meilleur classement dans la compétition.
- Analyser les erreurs et présenter les limites des méthodes utilisées.

2. Périmètre du projet

Les objectifs du projet sont les suivants :

- Extraire le texte des documents PDF et préparer le corpus pour l'analyse.
- Nettoyer et tokeniser les textes.
- Calculer les fréquences des mots et des séquences de mots.
- Générer une liste préliminaire d'entités candidates.
- Filtrer les candidats pour identifier les personnages et, si nécessaire, les lieux.
- Reconnaître les mentions de personnages dans le texte.
- Regrouper les différentes appellations d'un même personnage.
- Détecter les interactions entre personnages à partir de leurs cooccurrences dans le texte.
- Construire des graphes de personnages pour les livres du corpus principal.
- Générer un graphe distinct pour chaque chapitre du corpus d'évaluation.
- Exporter les graphes au format GraphML et préparer le fichier CSV attendu pour Kaggle.
- Évaluer les résultats avec les scores F1 des nœuds et des arêtes.
- Améliorer progressivement les algorithmes afin d'obtenir un meilleur classement dans la compétition.
- Analyser les erreurs et présenter les limites des méthodes utilisées.

2.1. Fonctionnalités attendues

Le programme doit proposer les fonctionnalités suivantes :

1. **Extraction du texte :** convertir les documents PDF en fichiers texte exploitables.
2. **Nettoyage :** corriger autant que possible les espaces incorrects, les coupures de mots et les erreurs issues de l'extraction.
3. **Tokenisation :** découper les textes en unités lexicales.
4. **Suppression des mots vides :** utiliser une liste de mots fréquents peu informatifs pour certaines analyses.
5. **Calcul des n-grammes :** extraire les unigrammes, bigrammes et trigrammes et calculer leurs fréquences.
6. **Génération des candidats :** construire une liste initiale d'entités nommées à partir de critères lexicaux et statistiques.

7. **Filtrage :** réduire les candidats non pertinents et produire des listes de personnages et, si nécessaire, de lieux.
8. **Reconnaissance des personnages :** détecter les mentions de personnages dans les textes, à l'aide de règles, de méthodes statistiques ou d'outils de reconnaissance d'entités nommées.
9. **Résolution des alias :** associer les différentes formes d'un même nom à un personnage unique.
10. **Détection des interactions :** considérer deux personnages comme étant en interaction lorsque leurs mentions apparaissent à une distance maximale de 25 tokens, conformément à la définition du sujet.
11. **Construction des graphes :** créer un nœud par personnage identifié et une arête entre deux personnages lorsqu'une interaction est détectée. Le poids de l'arête correspond au nombre de cooccurrences retenues.
12. **Graphe par livre :** construire et examiner les réseaux de personnages du corpus principal.
13. **Graphe par chapitre :** générer un graphe indépendant pour chaque chapitre du corpus d'évaluation de Kaggle.
14. **Exportation :** enregistrer les graphes au format GraphML et produire le fichier CSV de soumission selon le format imposé.
15. **Évaluation :** analyser les erreurs, mesurer la qualité des graphes et exploiter les scores de la compétition pour améliorer les résultats.

2.2. Technologies utilisées

- **Langage de programmation :** Python.
- **Environnement de développement :** Visual Studio Code et, si nécessaire, un notebook Python.
- **Traitement des textes :** expressions régulières, tokenisation, fréquences et n-grammes.
- **Reconnaissance d'entités nommées :** méthodes heuristiques et outils de TALN pouvant être comparés selon leurs résultats.
- **Construction des graphes :** NetworkX.
- **Formats de données :** PDF, TXT, JSON, GraphML et CSV.
- **Plateforme d'évaluation :** Kaggle, pour soumettre les graphes par chapitre et comparer les performances.
- **Documentation :** rapport scientifique et documentation du code.

Le choix définitif des méthodes de reconnaissance des personnages et de résolution des alias sera effectué en fonction de leur efficacité, de leur complexité et des résultats obtenus sur les données disponibles.

3. Public Cible

3.1. Profil des utilisateurs

Les utilisateurs visés sont principalement entre l'âge de 18 - 45 ans :

- Les étudiants et chercheurs travaillent sur l'analyse de textes littéraires.
- Les personnes intéressées par le traitement automatique du langage naturel.

- Les utilisateurs souhaitant explorer les personnages et leurs interactions dans un récit.
- Les personnes souhaitant visualiser et analyser des réseaux de personnages.

Les résultats devront être suffisamment clairs pour permettre de comprendre les personnages identifiés, leurs différentes appellations et les relations représentées dans les graphes.

3.2. Nombre d'utilisateurs pour les tests

Une première phase de tests exploratoires pourra être réalisée avec **3 à 5 utilisateurs**, par exemple des étudiants du même parcours.

Les tests porteront sur la facilité d'exécution des programmes, la compréhension des résultats, la lisibilité des graphes et l'interprétation des relations détectées.

4. Scénarios de Tests Utilisateurs

4.1. Objectifs des tests utilisateurs

Les tests auront pour objectifs de :

- Vérifier que les programmes s'exécutent correctement.
- Évaluer la pertinence des personnages détectés.
- Vérifier que les différentes appellations d'un même personnage sont correctement regroupées.
- Contrôler la cohérence des relations entre personnages.
- Évaluer la lisibilité des graphes.
- Vérifier que les fichiers GraphML et CSV sont générés dans les formats attendus.
- Comprendre les erreurs observées dans les résultats de la compétition Kaggle.
- Recueillir des suggestions d'amélioration.

4.2. Méthodologie de tests

Les tests auront pour objectifs de :

- Vérifier que les programmes s'exécutent correctement.
- Évaluer la pertinence des personnages détectés.
- Vérifier que les différentes appellations d'un même personnage sont correctement regroupées.
- Contrôler la cohérence des relations entre personnages.
- Évaluer la lisibilité des graphes.
- Vérifier que les fichiers GraphML et CSV sont générés dans les formats attendus.
- Comprendre les erreurs observées dans les résultats de la compétition Kaggle.
- Recueillir des suggestions d'amélioration.

4.3. Scénarios de tests

- **Scénario 1 :** exécuter les scripts de préparation du corpus et vérifier que les fichiers attendus sont générés.

- **Scénario 2 :** consulter la liste des entités candidates et identifier des éléments pertinents ou incorrects.
- **Scénario 3 :** comparer les listes avant et après filtrage.
- **Scénario 4 :** vérifier la détection de personnages dans un extrait du corpus.
- **Scénario 5 :** vérifier que plusieurs appellations d'un même personnage sont regroupées correctement.
- **Scénario 6 :** contrôler les interactions détectées entre deux personnages à partir de la règle de cooccurrence.
- **Scénario 7 :** générer un graphe pour un livre et examiner les personnages les plus connectés.
- **Scénario 8 :** générer un graphe pour chaque chapitre du corpus d'évaluation.
- **Scénario 9 :** vérifier que les fichiers GraphML contiennent les attributs requis et que le CSV de soumission respecte le format Kaggle.
- **Scénario 10 :** soumettre une première version sur Kaggle, relever le score F1 et identifier les améliorations possibles.

5. Critères de Réussite et Évaluations

5.1. Indicateurs de performance (KPI)

Les indicateurs suivants seront utilisés pour évaluer le projet :

- **Exécution :** les scripts s'exécutent sans erreur bloquante.
- **Production des résultats :** les fichiers intermédiaires et les graphes attendus sont générés.
- **Qualité des entités :** proportion de personnages pertinents dans un échantillon vérifié.
- **Résolution des alias :** capacité à regrouper les différentes appellations d'un même personnage sans fusionner des personnages distincts.
- **Qualité des relations :** pertinence des interactions détectées.
- **Conformité technique :** validité des fichiers GraphML et du CSV de soumission.
- **Score F1 des nœuds :** mesure de la qualité de l'identification des personnages.
- **Score F1 des arêtes :** mesure de la qualité des relations entre personnages.
- **Score Kaggle :** suivi du score global et du classement de l'équipe sur le leaderboard.
- **Lisibilité :** facilité d'interprétation des graphes.
- **Satisfaction utilisateur :** note de satisfaction recueillie lors des tests.

Le score de la compétition repose sur les performances relatives aux nœuds et aux arêtes. L'objectif est d'améliorer ces deux composantes sans dégrader excessivement l'une au profit de l'autre.

Les seuils de performance seront fixés après l'établissement d'une première version de référence. L'objectif sera ensuite d'améliorer progressivement les résultats grâce à des tests et à des ajustements mesurés.

5.2. Feedback des utilisateurs

Les retours seront recueillis au moyen d'observations, d'un questionnaire court et de l'analyse des résultats obtenus. Chaque amélioration importante devra être testée afin de déterminer si elle augmente effectivement la qualité des graphes. Les changements seront documentés pour pouvoir comparer les versions de l'algorithme et comprendre l'origine des gains ou des pertes de performance.

6. Contraintes et Risques

6.1. Contraintes techniques

- La qualité des résultats dépend de la qualité du texte extrait des PDF.
- Les coupures de mots et les erreurs de tokenisation peuvent nuire à la reconnaissance des personnages.
- Les méthodes heuristiques peuvent confondre personnages, lieux et mots ordinaires.
- Les outils génériques de reconnaissance d'entités nommées peuvent ne pas reconnaître certains noms fictifs.
- La résolution des alias peut fusionner des personnages distincts ou séparer les différentes formes d'un même nom.
- La règle de cooccurrence peut créer des relations qui ne correspondent pas nécessairement à une interaction directe dans le récit.
- Les graphes volumineux peuvent être difficiles à visualiser.
- Les fichiers produits doivent respecter le format demandé par Kaggle.
- Les améliorations doivent être évaluées sans utiliser de manière incorrecte les données réservées à l'évaluation.

6.2. Risques potentiels

- **Erreurs de reconnaissance :** des personnages peuvent être oubliés ou des mots ordinaires considérés comme des noms.
- **Erreurs de résolution des alias :** plusieurs personnages peuvent être confondus.
- **Relations imprécises :** des cooccurrences peuvent produire des arêtes non pertinentes.
- **Score Kaggle insuffisant :** la première version peut obtenir un score faible, ce qui nécessitera plusieurs améliorations.
- **Erreurs de format :** un fichier GraphML ou CSV incorrect peut empêcher la soumission.
- **Retards :** le nettoyage du corpus et l'amélioration des algorithmes peuvent prendre davantage de temps que prévu.
- **Droits d'utilisation :** les textes du corpus ne doivent pas être diffusés si les droits applicables ou les consignes du cours l'interdisent.

7. Planning Prévisionnel

7.1. Phases du projet

- **Déjà fait phase 1 Préparation du corpus :** extraction, nettoyage et tokenisation.
- **Déjà fait phase 2 Extraction des candidats :** calcul des fréquences et des n-grammes, génération de la liste d'entités.

- **Déjà fait phase 3 Filtrage :** amélioration des listes de personnages et, si nécessaire, de lieux.
- **phase 4 Première version des graphes :** détection des mentions, résolution des alias, détection des cooccurrences et construction des graphes.
- **Phase 5 Première soumission Kaggle :** génération des graphes par chapitre, exportation en GraphML et préparation du CSV.
- **Phase 6 Amélioration des performances :** analyse des erreurs, comparaison des méthodes et optimisation du score F1.
- **Phase 7 Validation :** vérification des graphes du corpus principal, contrôle des fichiers et tests utilisateurs.
- **Phase 8 Documentation et présentation :** rédaction de l'article, documentation du code et préparation de la présentation finale.

7.2. Livrables

- Les scripts Python de préparation et d'analyse du corpus.
- Les fichiers intermédiaires nécessaires au traitement.
- Les listes préliminaires et filtrées d'entités.
- Les statistiques du corpus.
- Les graphes de personnages générés pour les livres du corpus principal.
- Les graphes par chapitre au format GraphML pour la compétition.
- Le fichier CSV de soumission Kaggle conforme au format demandé.
- Les résultats des évaluations et le suivi des scores obtenus sur Kaggle.
- Le code source documenté, accompagné d'un README expliquant son installation et son utilisation.
- Un article scientifique de quatre pages suivant le format demandé dans le sujet.
- Une synthèse des tests, des erreurs observées et des améliorations réalisées.
- Le présent cahier des charges et les supports de présentation.

8. Équipe du Projet

Le projet réalisé par moi-même.

9. Annexes

9.1. Exemple de questionnaire utilisateur

1. Avez-vous réussi à exécuter les programmes sans aide ?
2. Les personnages détectés vous semblent-ils pertinents ?
3. Comprenez-vous comment les différentes appellations d'un personnage sont regroupées ?
4. Les relations représentées dans les graphes vous semblent-elles cohérentes ?
5. Le graphe permet-il de repérer les personnages les plus connectés ?
6. Les résultats sont-ils faciles à comprendre ?
7. Avez-vous remarqué des personnages manquants ou des relations incorrectes ?
8. Quelles améliorations proposeriez-vous ?
9. Quelle note de satisfaction donneriez-vous à l'outil, de 1 à 5 ?

9.2. Règles principales de génération des graphes

- Chaque nœud représente un personnage identifié.
- L'attribut `names` d'un nœud contient les différentes appellations reconnues pour ce personnage.
- Une arête est créée lorsque les mentions de deux personnages satisfont la règle de cooccurrence du sujet, avec une distance maximale de 25 tokens.
- Le poids d'une arête correspond au nombre de cooccurrences retenues.
- Un graphe est généré pour chaque livre étudié dans le corpus principal et pour chaque chapitre du corpus d'évaluation.
- Les graphes destinés à Kaggle sont exportés au format GraphML et référencés dans le CSV de soumission selon le format imposé par la compétition.

9.3. Outils envisagés

- Python et Visual Studio Code pour le développement.
- Les bibliothèques de traitement de texte et de reconnaissance d'entités nommées.
- NetworkX pour la construction et l'exportation des graphes.
- Matplotlib pour certaines visualisations.
- Kaggle pour la soumission et l'évaluation des résultats.
- GitHub ou GitLab pour la gestion du code source.